Replication package for "Fast Solution of Dynamic Intra-Household Bargaining Models"

Adam Hallengreen, Thomas H. Jørgensen and Annasofie M. Olesen

Submitted to *Quantitative Economics*. The code is also available at https://github.com/ThomasHJorgensen/FastBargaining.

Overview

The code in this package solves, simulates and calibrates the dynamic household models in the paper. It is written in Python, and the solution and simulation of the application model run in C++ that Python compiles with Microsoft Visual C++. No microdata are used. The application model is calibrated to 13 published statistics, which are typed directly into the code (see the data availability statements below).

A single master notebook, `run_all.ipynb`, runs six notebooks in sequence. Together they generate all figures and tables in the paper and the Supplemental Material that come from computations: Figures 1–4, Tables 2–4 and 6–8, and Figure S1. Tables 1, 5 and S1 list parameter values that are inputs to the code, and Figure S2 is a stylized drawing made directly in LaTeX.

The full run took about 33 hours on a 104-core server with 1.5 TB of RAM. The four notebooks for the application model need between 300 GB and 2 TB of memory (RAM plus page file), so they cannot be run on a desktop computer. The two notebooks for the illustrative model (Figure 1 and Tables 2–3) and for Figure S1 run on an ordinary computer. See Computational requirements.

Data Availability and Provenance Statements

- [x] This paper does not involve analysis of external data (i.e., no data are used or the only data are generated by the authors via simulation in their code).

The paper does not use microdata. All data analyzed in the paper are simulated by the code from the models. The only empirical inputs are the 13 calibration targets of the application model (paper Table 6). They are published summary statistics from three journal articles and from the U.S. Bureau of Labor Statistics (BLS), and they are typed into `Application/01. Calibration.ipynb` (section "Empirical Moments to Match"). The package contains no data files. For transparency, the sources of the 13 targets are documented below.

Statement about Rights

- [ ] I certify that the author(s) of the manuscript have legitimate access to and permission to use the data used in this manuscript.
- [ ] I certify that the author(s) of the manuscript have documented permission to redistribute/publish the data contained within this replication package. Appropriate permission are documented in the LICENSE.txt file.

Not applicable. No external data are used, and the package contains no data files. The 13 calibration targets are published summary statistics, which are quoted with citations.

License for Data

Not applicable, as the package contains no data files. The README, the generated output files and the calibrated parameter file are licensed under CC BY 4.0, see `LICENSE.txt`.

Summary of Availability

Not applicable, as no external data are used. All 13 calibration targets are published in the sources below. Two of the journal articles require a subscription, but earlier versions of both are freely available as NBER working papers.

Details on each Data Source

The paper uses no data sources. This section documents the published sources of the 13 calibration targets and of the tax schedules used in the model.

Data source	Calibration targets (paper Table 6)	Value(s) in code	Provided	Citation
Eckstein, Keane and Lifshitz (2019), Supplemental Appendix J	Wages of women and men aged 25–34 and 35–41; employment rates of women and men aged 35–41; share married aged 35–41	40.1, 49.3, 50.4, 67.8; 64.0, 88.0; 69.0	Values typed into code	Eckstein, Keane, and Lifshitz (2019a, 2019b)
Blundell, Pistaferri and Saporta-Eksten (2018), Table 1	Total household consumption	42.716 (thousand USD)	Value typed into code	Blundell, Pistaferri, and Saporta-Eksten (2018)
Meyer and Sullivan (2023), Table 2	90:10 percentile ratio of consumption	3.33 × 1.0954	Value typed into code	Meyer and Sullivan (2023)
American Time Use Survey (ATUS), 2024 annual averages, BLS news release Table 3	Working hours and home production hours of women and men aged 35–44	4.41 and 5.70 hours/day (work); 2.23+0.75+1.47+0.08 and 1.60+0.54+0.88+0.10 hours/day (home production)	Values typed into code	U.S. Bureau of Labor Statistics (2025)
Bronson, Haanwinckel and Mazzocco (2024), Table A.1: U.S. federal income tax schedules for 2000 (Supplemental Material, Table S1)	None. Tax brackets and rates used in the model's budget constraints	Brackets and rates in `Application/cppfuncs/couple.cpp` (lines 30–31) and `single.cpp` (lines 30, 32)	Values typed into code	Bronson, Haanwinckel, and Mazzocco (2024)

Eckstein, Keane and Lifshitz (2019). We use the "Data" column for the 1975 birth cohort in Tables J-I (wages and employment) and J-II (marriage rate) of the article's Supplemental Material. Wages are real annual wages of full-time, full-year married workers in thousands of 2012 USD. Employment is the share of the population working at least 10 hours per week. The 1975 cohort is observed only until age 41, so the "ages 35–44" cell covers ages 35–41. The article and its Supplemental Material are on the Econometrica website (https://doi.org/10.3982/ECTA14474). Access is free.

Blundell, Pistaferri and Saporta-Eksten (2018). The target is the sample mean of total household consumption in Table 1, panel B: 42,716 USD in 2010 prices, excluding durables. The sample is married couples in the Panel Study of Income Dynamics (PSID) in which both spouses work. The article is in the *Journal of Political Economy* (https://doi.org/10.1086/698752), which requires a subscription or institutional access. An earlier version is freely available as NBER Working Paper 24006.

Meyer and Sullivan (2023). The target is the 2017 level of the 90/10 ratio of well-measured consumption. It is computed from Table 2 as the 1960–61 level (3.33) times one plus the 1961–2017 change (9.54 percent). Consumption is measured from the Consumer Expenditure Survey at the family level, equivalized with the NAS scale, and weighted by persons. The values are from the published article (*Journal of Political Economy*, 131 (2), February 2023, Table 2, p. 261, https://doi.org/10.1086/721702), which requires a subscription or institutional access. The revised NBER Working Paper 23655 (January 2022, https://www.nber.org/papers/w23655) is freely available and reports the same values in its Table 2.

American Time Use Survey (ATUS). The targets are average hours per day in primary activities for women and men aged 35–44 in 2024. They average over all persons in the age group and all days of the week. They are taken from Table 3, "Time spent in primary activities for the civilian population by age, sex, race, Hispanic or Latino ethnicity, marital status, and educational attainment, 2024 annual averages", of the BLS news release "American Time Use Survey – 2024 Results" (USDL-25-1060, June 26, 2025): https://www.bls.gov/news.release/archives/atus_06262025.pdf. The values are in the rows "35 to 44 years" under "Men, 15 years and over" and "Women, 15 years and over" (page 11). Market work is "Working and work-related activities". Home production is the sum of "Household activities", "Purchasing goods and services", "Caring for and helping household members" and "Caring for and helping nonhousehold members". The code converts hours per day to hours per week (× 365/52). The paper reports annual hours (× 365). The data are public domain and free to access without registration. The values were checked against the release on October 27, 2026.

Tax schedules. The model taxes gross labor income with the 2000 U.S. federal income tax brackets and marginal rates for singles and married couples (Supplemental Material, Section S3 and Table S1). These are as tabulated in Bronson, Haanwinckel, and Mazzocco (2024, Table A.1). They are hard-coded in the C++ files listed above.

Externally calibrated parameters. The parameters in paper Table 5 come from the studies cited there, not from data. They are set in `Application/DynamicTimeAllocationModel.py` (see the list of tables below).

Dataset list

The package contains no data files. It contains one intermediate file generated by the code:

File	Source	Notes	Provided
`Application/par_files/calibrated_par.pkl`	Generated by `Application/01.Calibration.ipynb`	Python pickle of the model's parameter dictionary at the calibrated parameters. It is read by notebooks 02–04, and its values are also written to `Application/output/calibration_parameters.tex`.	Yes

Computational requirements

Software Requirements

- [x] The replication package contains one or more programs to install all dependencies and set up the necessary directory structure.

The Python dependencies are installed from `pyproject.toml` with `uv.lock` and `.python-version`, from `requirements.txt` or from `environment.yml` (see the instructions to replicators). The output folders are part of the package.

- **Operating system: Windows (64-bit).** The code was last run on Windows Server 2022 Datacenter (version 10.0.20348). Windows is required because EconModel compiles the C++ code by writing and running a Windows batch file that calls the Microsoft Visual C++ compiler, and it loads the result as a `.dll`.
- **Microsoft Visual Studio Community 2022** (version 17.7.6, MSVC 14.37.32822) with the workload "Desktop development with C++". EconModel looks for `vcvarsall.bat` in `C:/Program Files/Microsoft Visual Studio/2022/Community/VC/Auxiliary/Build/` and in the corresponding folders of Visual Studio 2019 Community and Visual Studio 18 Community. Install the Community edition in its default location. Other editions are not found automatically.
- **NLopt 2.4.2**, 64-bit Windows DLL (Johnson 2014). It is included in `Application/cppfuncs/nlopt-2.4.2-dll64/`, together with the import library `libnlopt-0.lib`, which was generated with `lib /def:libnlopt-0.def /machine:x64`. The original archive is included as `Application/cppfuncs/nlopt-2.4.2-dll64.zip`.
- **Python 3.13.13** (CPython build distributed by uv), managed with **uv 0.11.8**. All Python packages are pinned in `uv.lock` (and in `requirements.txt` for pip). The main packages are:

- EconModel 0.17, installed from GitHub at commit `367f22048e49cad10cc54c3f09f5f313180cca6f` (https://github.com/NumEconCopenhagen/EconModel), so `git` must be installed
- consav 0.12
- numpy 2.4.4, numba 0.65.1, scipy 1.17.1, pandas 3.0.2
- matplotlib 3.10.9, seaborn 0.13.2
- papermill 2.7.0, ipykernel 7.2.0 (to execute the notebooks)

Compilation. The C++ code is compiled automatically by `Application/01. Calibration.ipynb` (`do_compile = True`) through EconModel. The resulting compiler call, run from `Application/` after `call vcvarsall.bat x64`, is

```
cl /LD /EHsc /O2 /openmp cppfuncs/solve.cpp setup_omp.cpp cppfuncs/nlopt-2.4.2-dll64/libnlopt-
         0.lib
```

It produces `Application/solve.dll`. `setup_omp.cpp` is a small file that EconModel writes and deletes on the fly. The files `par_struct.cpp`, `sol_struct.cpp` and `sim_struct.cpp` in `Application/cppfuncs/` are written automatically by EconModel from the Python model, and `solve.cpp` includes the remaining C++ files. A compiled `Application/solve.dll` is included, so notebooks 02–04 can also run without a compiler.

Controlled Randomness

- [x] Random seed is set at line 73 of program `Illustration/UnitaryModel.py` and at line 150 of program `Application/DynamicTimeAllocationModel.py`
- [ ] No Pseudo random generator is used in the analysis described here.

All random numbers are drawn in Python with NumPy's global generator before the C++ code is called. The C++ code itself draws no random numbers, and its parallel loops write to separate array elements, so the number of OpenMP threads should only affect the run time.

- Illustrative model: `Illustration/UnitaryModel.py`, line 73 (`par.seed = 9210`), used at line 147 (`np.random.seed(par.seed)`). In `Illustration/01. UnitaryHousehold.ipynb` (section "Accuracy measures"), Monte Carlo run `i_mc` sets `par.seed = i_mc`.
- Application model: `Application/DynamicTimeAllocationModel.py`, line 150 (`par.seed = 9210`), used at line 647 (`np.random.seed(par.seed)`). Shocks are redrawn from this seed for every evaluation of the calibration objective. In `Application/03. Timings.ipynb`, Monte Carlo run `i_mc` sets `par.seed = i_mc`.

Memory, Runtime, Storage Requirements

Summary

Approximate time needed to reproduce the analyses on a standard 2026 desktop machine:

- [ ] <10 minutes
- [ ] 10-60 minutes
- [ ] 1-2 hours
- [ ] 2-8 hours
- [ ] 8-24 hours
- [x] 1-3 days
- [ ] 3-14 days
- [ ] > 14 days

Approximate storage space needed:

- [ ] < 25 MBytes

- [ ] 25 MB - 250 MB
- [ ] 250 MB - 2 GB
- [ ] 2 GB - 25 GB
- [ ] 25 GB - 250 GB
- [x] > 250 GB
- [x] Not feasible to run on a desktop machine, as described below.

The run time refers to the server described below. The storage is dominated by the page file. The package itself takes about 30 MB and the Python environment about 0.5 GB. However, `03. Timings` needs about 2 TB of memory, so on a machine with 1.5 TB of RAM, Windows needs a page file of about 400 GB on disk, and more on a machine with less RAM (see below). An automatically managed page file grows to at most one eighth of the size of its drive, so on a smaller drive the page file size must be set by hand.

Details

The code was last run on a Dell PowerEdge R760 server with two Intel Xeon Platinum 8470 processors (2.0 GHz, 104 cores in total), 1.5 TB of RAM and Windows Server 2022 Datacenter. The C++ code runs in parallel with 64 OpenMP threads.

The solution arrays of the application model are large. One baseline model takes about 300 GB of memory, and the dense "true" model in `03. Timings` takes about 1.7 TB. On the authors' server, the part of the memory use above the 1.5 TB of physical RAM was covered by Windows' automatically managed page file. The memory use below is approximate and computed from the sizes of the allocated arrays.

Notebook (in the order run by `run_all.ipynb`)	Wall-clock time, last run	Approx. peak memory
`Illustration/01. UnitaryHousehold.ipynb`	2 h 41 min	< 4 GB (single core)
`Application/01. Calibration.ipynb`	19 h 31 min	300 GB
`Application/02. Comparisons.ipynb`	2 h 16 min	1.4 TB (four models with about 350 GB each are kept in memory)
`Application/03. Timings.ipynb`	8 h 2 min	2.0 TB (true model 1.7 TB plus one model of 0.3 TB)
`Application/04. Counterfactuals.ipynb`	21 min	1.2–1.5 TB (four models of 0.3 TB)
`Appendix/Illustrate bargaining algo.ipynb`	1 min	< 1 GB
Total	**32.9 h**	

The calibration in `01. Calibration` takes most of the time: each evaluation of the objective function solves and simulates the model, and the Nelder–Mead search stops after 500 evaluations. The other notebooks use the calibrated parameters in `Application/par_files/calibrated_par.pkl`.

Parts that run on an ordinary computer. `Illustration/01. UnitaryHousehold.ipynb` (Figure 1 and Tables 2–3) and `Appendix/Illustrate bargaining algo.ipynb` (Figure S1) are written in Python only. They do not use the C++ code or Visual Studio, run on a single core and need less than 4 GB of memory, so they can be run on an ordinary computer. On the server above they took 2 h 41 min and 1 min. To run them without the rest of the package, set up the Python environment (steps 2–3 of the instructions below) and run them from the root of the package:

```
.venv\Scripts\python -m papermill "Illustration/01. UnitaryHousehold.ipynb" "Illustration/01.
        UnitaryHousehold.ipynb" --cwd Illustration
.venv\Scripts\python -m papermill "Appendix/Illustrate bargaining algo.ipynb"
        "Appendix/Illustrate bargaining algo.ipynb" --cwd Appendix
```

The four notebooks in `Application/` (Figures 2–4, Tables 4 and 6–8 and footnote 25) need between 300 GB and 2 TB of memory and cannot be run on an ordinary computer.

Description of programs/code

- `run_all.ipynb`: the master notebook. It runs the six notebooks below in order with papermill. Each notebook is run in its own folder. An executed notebook replaces the original only if it completes without error.
- `Illustration/`: the illustrative two-person unitary model in Section 2 of the paper.
  - `UnitaryModel.py`: model class with VFI, analytical EGM, numerical EGM and iEGM solvers, simulation and accuracy measures.
  - `InterpolationFunctions.py`: helper functions for numerical inversion and interpolation.
  - `01. UnitaryHousehold.ipynb`: generates Figure 1 and Tables 2–3.
- `Application/`: the quantitative household model in Section 4 of the paper.
  - `DynamicTimeAllocationModel.py`: model class (`HouseholdModelClass`). It sets parameters and grids, allocates memory, draws shocks, calls the C++ solver and simulator, and computes moments, the calibration objective and the accuracy measures.
  - `cppfuncs/`: C++ code. `solve.cpp` holds the entry points (`solve`, `simulate`, `accuracy_measures`). `precompute.cpp` holds the intra-temporal precomputation and the iEGM interpolator, `single.cpp` and `couple.cpp` the problems of singles and couples (including the tax functions), and `bargaining.cpp` the limited commitment bargaining protocol. `simulate.cpp` simulates the model, and `index.cpp`, `tools.cpp`, `utils.cpp` and `logs.cpp` are helpers. `nlopt-2.4.2-dll64/` contains NLopt.
  - `01. Calibration.ipynb`: calibrates the model, writes Tables 6 and 7, compiles the C++ code and saves `par_files/calibrated_par.pkl`.
  - `02. Comparisons.ipynb`: plots policy functions and simulations from iEGM, numerical EGM and VFI side by side. These are diagnostic figures that stay inside the notebook. No output from this notebook appears in the paper.
  - `03. Timings.ipynb`: solves the dense "true" model and computes timing and accuracy (Table 4 and footnote 25).
  - `04. Counterfactuals.ipynb`: shows the iEGM interpolator (Figure 2) and runs the productivity-shock simulations (Figures 3–4 and Table 8).
  - `figure_manager.py`, `plots.py`: plotting helpers.
  - `par_files/calibrated_par.pkl`: calibrated parameters, see the dataset list.
- `Appendix/`: `Illustrate bargaining algo.ipynb` generates Figure S1 of the Supplemental Material. `figure_manager.py` is the same plotting helper as in `Application/`, with larger fonts.
- `pyproject.toml`, `uv.lock`, `.python-version`, `requirements.txt`, `environment.yml`: Python environment.
- `DEVELOPMENT.md`: how to set up the Python environment with uv, pip or conda, and how to update the dependency files. It is not needed for the replication.

License for Code

The code, including the compiled library `Application/solve.dll`, is licensed under a MIT license. See LICENSE.txt for details. The README, the output files and all other files written by the authors (such as `Application/par_files/calibrated_par.pkl`) are licensed under CC BY 4.0, and the third-party library NLopt keeps its own license.

Instructions to Replicators

All paths are relative to the root of the package. Commands are for the Windows command prompt.

1. Install Visual Studio Community 2022 with the workload "Desktop development with C++" in its default location.

2. Install `git` and `uv` (https://docs.astral.sh/uv/getting-started/installation/). `uv` is not needed if pip or conda is used instead (see step 3).

3. Open a command prompt in the root of the package and set up the Python environment with uv:

   ```
   uv python install 3.13.13
   uv sync --frozen --python 3.13.13
   ```

   This creates `.venv` with the exact package versions in `uv.lock`.

   Alternatively, use pip or conda. Both install the same package versions from `requirements.txt`, which is exported from `uv.lock`:

   - pip: install Python 3.13.13 from https://www.python.org/downloads/ and run

     ```
     py -3.13 -m venv .venv
     .venv\Scripts\python -m pip install -r requirements.txt
     ```

   - conda: run

     ```
     conda env create -f environment.yml
     conda activate fastbargaining
     ```

     This installs Python 3.13 from conda-forge and then the packages in `requirements.txt` with pip.

   `DEVELOPMENT.md` describes the three options in more detail.

4. Optional: if the machine has fewer than 64 hardware threads, set `threads` in the first code cell of `Application/01. Calibration.ipynb`. Notebooks 02–04 take the number of threads from `calibrated_par.pkl`, which notebook 01 writes.

5. Run the master notebook from the root of the package:

   ```
   .venv\Scripts\python -m papermill run_all.ipynb run_all.ipynb
   ```

   With conda, run `python -m papermill run_all.ipynb run_all.ipynb` in the activated `fastbargaining` environment instead. Alternatively, open `run_all.ipynb` in VS Code or JupyterLab, choose the `.venv` kernel (or the `fastbargaining` environment with conda) and run all cells.

6. `run_all.ipynb` prints the run time of each notebook. If a notebook fails, it prints `failed` followed by the error, and the original notebook is left unchanged. The outputs are written to `Illustration/output/`, `Application/output/` and `Appendix/output/`. The executed notebooks also contain all printed results.

Details

- Notebooks 02–04 can be run on their own (from their folder) with the included `calibrated_par.pkl` and `solve.dll`. If `01. Calibration` is run, it overwrites both files, and notebooks 02–04 must be run afterwards.

- The Nelder–Mead search in `01. Calibration` is capped at 500 evaluations of the objective function. The warning "Maximum number of function evaluations has been exceeded" is expected.
- The timing columns of Tables 2–4 depend on the hardware and on other load on the machine. All other numbers are deterministic given the software versions above.
- If the C++ code fails to link to NLopt, delete the folder `Application/cppfuncs/nlopt-2.4.2-dll64/` and set `do_reinstall_nlopt = True` in the first code cell of `01. Calibration.ipynb`. This unpacks the included archive `nlopt-2.4.2-dll64.zip` and regenerates `libnlopt-0.lib`.

List of tables and programs

The provided code reproduces:

- [x] All numbers provided in text in the paper
- [x] All tables and figures in the paper
- [ ] Selected tables and figures in the paper, as explained and justified below.

Tables 1, 5 and S1 report inputs to the code, and Figure S2 is a stylized LaTeX drawing. All other exhibits are generated by the code. Figures with several panels are saved as one file per panel (`*_subplot_1.pdf`, `*_subplot_2.pdf`, …, in the order (a), (b), …) and as one file with all panels (the same name without `_subplot_k`).

Exhibit	Program	Location in program	Output file	Note
Table 1	`Illustration/UnitaryModel.py`	lines 28–51 (unrestricted model), lines 151–159 (restrictions)	n.a.	Parameter inputs
Figure 1 (a), (b)	`Illustration/01. UnitaryHousehold.ipynb`	section "Illustrate the pre-computation of consumption interpolator"	`Illustration/output/illustration_iEGM_interp_subplot_1.pdf`, `..._subplot_2.pdf`	
Table 2	`Illustration/01. UnitaryHousehold.ipynb`	cell "# write to latex file"	`Illustration/output/MC_unitary.tex`	Timings depend on hardware
Table 3	`Illustration/01. UnitaryHousehold.ipynb`	cell "# write to latex file"	`Illustration/output/MC_unitary_pre.tex`	Timings depend on hardware
Figure 2 (a), (b)	`Application/04. Counterfactuals.ipynb`	section "Visualize precomputed consumption"	`Application/output/application_iEGM_interp_subplot_1.pdf`, `..._subplot_2.pdf`	
Table 4	`Application/03. Timings.ipynb`	cell "# write to latex file"	`Application/output/MC_limited.tex`	Timings depend on hardware
Table 5	`Application/DynamicTimeAllocationModel.py`	lines 47 (r), 48 (β), 60 (ρ), 74 (ζ), 75 (ω), 91 (δ), 103 (σ_ε)	n.a.	Parameter inputs
Table 6	`Application/01. Calibration.ipynb`	cell "# write calibration results to latex files"	`Application/output/calibration_fit.tex`	
Table 7	`Application/01. Calibration.ipynb`	cell "# write calibration results to latex files"	`Application/output/calibration_parameters.tex`	
Figure 3 (a)–(d)	`Application/04. Counterfactuals.ipynb`	section "Shock to low types human capital"	`Application/output/shock_Klow_subplot_1.pdf` … `_subplot_4.pdf`	
Figure 4 (a)–(d)	`Application/04. Counterfactuals.ipynb`	section "Shock to high types human capital"	`Application/output/shock_Khigh_subplot_1.pdf` … `_subplot_4.pdf`	

Exhibit	Program	Location in program	Output file	Note
Table 8	`Application/04. Counterfactuals.ipynb`	section "Table"	`Application/output/consumption_inequality.tex`	
Figure S1	`Appendix/Illustrate bargaining algo.ipynb`	section "Illustrate interpolation"	`Appendix/output/showcase_interpolation.pdf`	Supplemental Material
Figure S2	n.a.	n.a.	n.a.	Stylized illustration drawn in LaTeX
Table S1	`Application/cppfuncs/couple.cpp`, `single.cpp`	lines 30–31 and lines 30, 32	n.a.	Tax schedule inputs

Numbers in the text.

- Speed and accuracy comparisons in the abstract, the introduction, Section 2.3 and the conclusion (e.g. "15–20 times faster", "6% of the time", "around 19% of household expected income", "4.6% of the time", "0.167 vs. 0.765 seconds") are computed from Tables 2 and 3.
- The timings in Section 4.2.1, the introduction and the conclusion ("around 45 minutes", "40 minutes", "a couple of minutes", "about 20 times faster") are from Table 4.
- Footnote 25 (couples bargain in 0.85% of the periods) is written to `Application/output/share_power_updated.txt` by `Application/03. Timings.ipynb` (section "Print the fraction of points where power is updated in the simulation").
- The state space sizes in Section 4.2.1 (5,808,000 points) and footnote 24 (42,240,000 points) are products of the grid sizes: 4·4·11·11·10·10·30 and 4·4·11·15·20·20·40. The grid sizes are in `calibrated_par.pkl` and in the "true" model settings in `03. Timings.ipynb`.

Other output files. `Appendix/output/bargaining*.pdf` and `Appendix/output/showcase_power_shift*.pdf` are illustrations that are not used in the paper or the Supplemental Material.

References

Blundell, Richard, Luigi Pistaferri, and Itay Saporta-Eksten (2018), "Children, time allocation, and consumption insurance." *Journal of Political Economy*, 126 (S1), S73–S115. https://doi.org/10.1086/698752.

Bronson, Mary Ann, Daniel Haanwinckel, and Maurizio Mazzocco (2024), "Taxation and household decisions: An intertemporal analysis." NBER Working Paper 32861, National Bureau of Economic Research.

Eckstein, Zvi, Michael Keane, and Osnat Lifshitz (2019a), "Career and family decisions: Cohorts born 1935–1975." *Econometrica*, 87 (1), 217–253. https://doi.org/10.3982/ECTA14474.

Eckstein, Zvi, Michael Keane, and Osnat Lifshitz (2019b), "Supplement to 'Career and family decisions: Cohorts born 1935–1975'." *Econometrica Supplemental Material*, 87. https://doi.org/10.3982/ECTA14474.

Johnson, Steven G. (2014), "The NLopt nonlinear-optimization package, version 2.4.2." https://github.com/stevengj/nlopt.

Meyer, Bruce D. and James X. Sullivan (2023), "Consumption and income inequality in the United States since the 1960s." *Journal of Political Economy*, 131 (2), 247–284. https://doi.org/10.1086/721702.

U.S. Bureau of Labor Statistics (2025), "American Time Use Survey – 2024 Results." News Release USDL-25-1060, June 26, 2025, Table 3 [dataset]. https://www.bls.gov/news.release/archives/atus_06262025.pdf, accessed October 27, 2026.